DEVELOPER_GUIDE

HelixCode Developer Guide

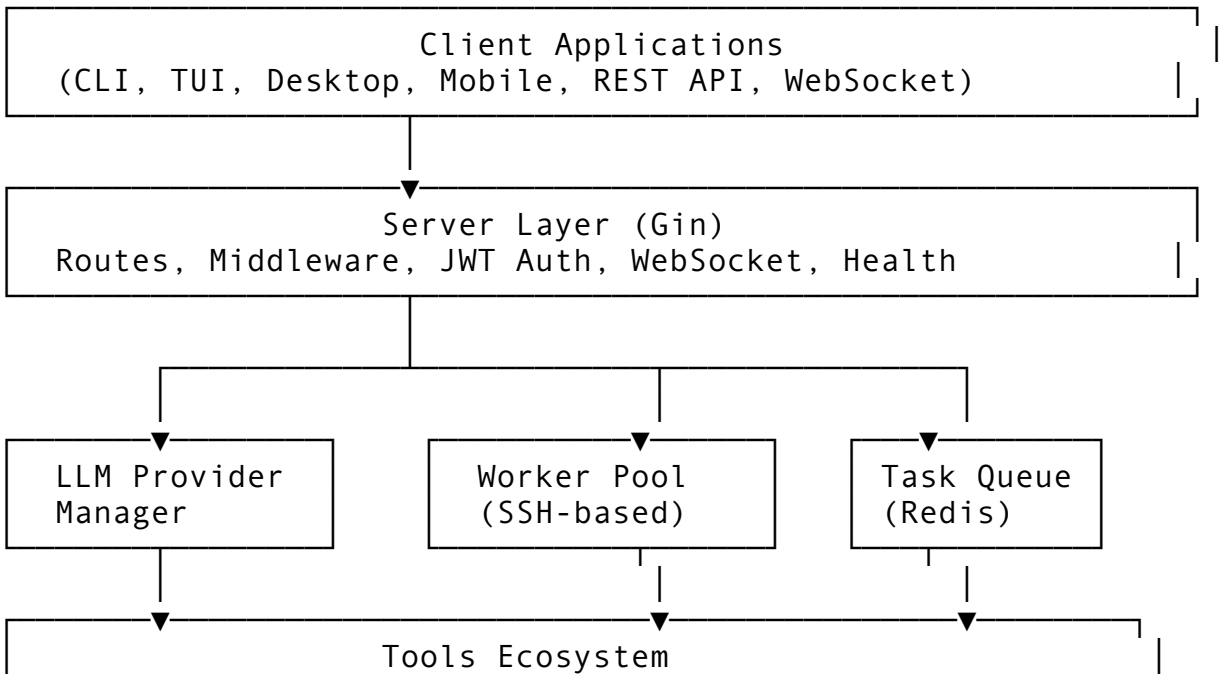
Version: 1.0.0
Date: 2026-05-08
Audience: Contributors and developers extending HelixCode

Table of Contents

Architecture Overview

HelixCode is a distributed AI development platform built in Go with a modular, plugin-based architecture.

Core Components



```
  (Filesystem, Shell, Web, Browser, Git, MCP, etc.)
```

Key Packages

Package	Purpose	Lines
`internal/llm`	Multi-provider LLM integration	5000+
`internal/worker`	SSH-based distributed worker pool	800+
`internal/task`	Task queue with checkpoints	1000+
`internal/server`	HTTP server with 50+ routes	1500+
`internal/tools`	40+ tools ecosystem	2000+
`internal/auth`	JWT authentication	470+
`internal/database`	PostgreSQL layer	600+
`internal/workflow`	Development workflow execution	1100+

Development Environment Setup

Prerequisites

- **Go**: 1.24.0+ (toolchain go1.24.9)
- **PostgreSQL**: 15+ (optional, for database features)
- **Redis**: 7+ (optional, for caching)
- **Docker/Podman**: For containerized builds
- **Git**: SSH-based workflows

Quick Start

```
# Clone repository (SSH only)
git clone git@github.com:HelixDevelopment/HelixCode.git
cd helix_code/HelixCode

# Install dependencies
make setup-deps

# Generate logo assets (required before first build)
make logo-assets

# Build binary
make build

# Run tests (unit tests only)
make test

# Run with minimal config (DB/Redis disabled)
./bin/helixcode --config config/minimal-config.yaml
```

Environment Variables

Required for Production:

```
export HELIX_DATABASE_PASSWORD="secure-password"
export HELIX_AUTH_JWT_SECRET="high-entropy-secret"
export HELIX_REDIS_PASSWORD="redis-password"
```

LLM Provider Keys (as needed):

```
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
export GEMINI_API_KEY="..."
# etc.
```

IDE Setup

Recommended: VS Code with Go extension, or GoLand.

Key settings: - Enable `gopls` language server - Configure `go.lintTool` to `golangci-lint` - Set `go.testFlags` to `["-v"]`

Project Structure

```
helix_code/
├── cmd/                        # Application entry points
│   ├── server/main.go          # HTTP server
│   ├── cli/main.go             # CLI client (flag-based)
│   ├── root.go                 # Cobra root command
│   ├── helix-config/           # Config management CLI
│   └── ...
│
├── internal/                   # Core packages (40+)
│   ├── llm/                    # LLM provider implementations
│   ├── worker/                 # SSH worker pool
│   ├── task/                   # Task management
│   ├── server/                 # HTTP server
│   ├── database/               # PostgreSQL layer
│   ├── tools/                  # Tool ecosystem
│   ├── auth/                   # Authentication
│   └── ...
│
├── applications/               # Platform-specific apps
│   ├── desktop/                # Fyne desktop app
│   ├── terminal-ui/            # tview TUI
│   ├── android/                # Android app
│   └── ios/                    # iOS app
│
├── api/                        # OpenAPI specification
│   └── openapi.yaml
│
```

```
├── config/                  # Configuration files
│   ├── config.yaml          # Primary config
│   ├── minimal-config.yaml # DB/Redis disabled
│   └── ...
│
├── tests/                   # Test framework
│   ├── e2e/challenges/      # Challenge-based E2E
│   └── automation/          # Hardware automation
│
└── docker/                  # Docker assets
```

Coding Standards

Go Conventions

1. **Formatting**: Use `go fmt` before every commit
2. **Linting**: Run `golangci-lint run ./...`
3. **Vet**: Run `go vet ./...`
4. **Comments**: Exported functions MUST have doc comments
5. **Error Handling**: No silent failures; explicit error returns

Project Conventions

1. **Package Layout**: All production code in `internal/` or `cmd/`
2. **Testing**: Table-driven tests with `t.Run()` subtests
3. **Mocking**: Mocks only in unit tests (`-short` flag)
4. **Build Tags**: `//go:build integration` for integration tests
5. **No Comments**: Zero `// TODO`, `// FIXME`, `// simulated` in production code

Example: Standard Package Structure

```go
// Package example demonstrates standard layout.
package example

import (
    "context"
    "fmt"
)

// Config holds configuration for Example.
type Config struct {
    Name string
}

// Service provides example functionality.
type Service struct {
    cfg *Config
}
```

```go
// New creates a new Service instance.
func New(cfg *Config) (*Service, error) {
    if cfg == nil {
        return nil, fmt.Errorf("config is required")
    }
    return &Service{cfg: cfg}, nil
}

// DoSomething performs an action.
func (s *Service) DoSomething(ctx context.Context) error {
    // Real implementation, no stubs
    return nil
}
```

Testing Strategy

Test Categories

1. **Unit Tests** (`-short` flag)
    - Mocks allowed
    - Fast execution
    - Test individual functions
2. **Integration Tests** (`-tags=integration`)
    - Real database/Redis
    - Real HTTP calls
    - No mocks
3. **E2E Challenges**
    - Full system tests
    - Real LLM API calls
    - End-user workflows
4. **Security Tests**
    - OWASP Top 10
    - Credential scanning
    - TLS enforcement

Running Tests

```bash
# Unit tests only (fast)
make test

# Integration tests (requires Docker)
make test-integration-full

# Full infrastructure tests
make test-infra-up
make test-complete
make test-infra-down

# Single test
```

```
go test -v -run TestName ./path/to/package

# Coverage
make test-coverage
```

Anti-Bluff Testing Rules

Per Article XI §11.9, every test PASS must guarantee:

1. **Quality**: Correct behavior under real inputs
2. **Completion**: Wired end-to-end, no stubs
3. **Usability**: Feature works for end users

Forbidden patterns: - `t.Skip()` without `SKIP-OK:` marker - Tests that assert only on `NotNil` without content verification - Hardcoded success values - Mocks in integration/E2E tests

---

LLM Provider Integration

Adding a New Provider

1. Create provider struct in `internal/llm/providers/<name>/`
2. Implement `Provider` interface:

```
type Provider interface {
    Generate(ctx context.Context, req *LLMRequest)
        (*LLMResponse, error)
    GenerateStream(ctx context.Context, req *LLMRequest)
        (<-chan StreamChunk, error)
    GetCapabilities() *ProviderCapabilities
    IsAvailable() bool
}
```

3. Register in `internal/llm/registry.go`
4. Add configuration in `config/config.yaml`
5. Write tests (unit + integration)
6. Add Challenge for provider

Provider Capabilities

```
type ProviderCapabilities struct {
    Name              string
    SupportsStreaming bool
    SupportsTools     bool
    SupportsVision    bool
    MaxTokens         int
```

```
    ContextWindow     int
}
```

Model Selection Strategy

Configured in `llm.selection.strategy`: - `performance`: Fastest provider wins - `cost`: Cheapest provider wins - `quality`: Highest-scored model wins - `round-robin`: Distribute load evenly

---

Tool Development

Tool Interface

All tools implement the `Tool` interface:

```
type Tool interface {
    Name() string
    Description() string
    Parameters() jsonschema.Schema
    Execute(ctx context.Context, params map[string]interface{})
        (interface{}, error)
}
```

Example: Simple Tool

```
package mytool

import (
    "context"
    "dev.helix.code/internal/tools"
)

type MyTool struct{}

func (t *MyTool) Name() string {
    return "my_tool"
}

func (t *MyTool) Description() string {
    return "Performs a specific action"
}

func (t *MyTool) Parameters() jsonschema.Schema {
    return jsonschema.Schema{
        Type: "object",
        Properties: map[string]jsonschema.Schema{
            "input": {Type: "string"},
        },
```

```
        Required: []string{"input"},
    }
}

func (t *MyTool) Execute(ctx context.Context, params
        map[string]interface{}) (interface{}, error) {
    input, ok := params["input"].(string)
    if !ok {
        return nil, fmt.Errorf("input must be string")
    }
    // Real implementation
    return map[string]string{"result": "processed: " + input},
        nil
}
```

Tool Categories

- **Filesystem**: `fs_read`, `fs_write`, `fs_edit`, `glob`, `grep`
- **Shell**: `shell`, `shell_background` (sandboxed)
- **Web**: `web_fetch`, `web_search`
- **Browser**: `browser_launch`, `browser_navigate`, `browser_screenshot`
- **Git**: Automation tools
- **Multi-edit**: Transactional multi-file editing
- **MCP**: Model Context Protocol integration

Security Considerations

Authentication

- **JWT**: HS256 signing, configurable expiry
- **Password Hashing**: bcrypt (cost 12) with argon2 fallback
- **Sessions**: Crypto-random tokens, IP binding optional

Authorization

- **Role-based**: admin, user, worker
- **Permission Engine**: Rule-based with deny-by-default
- **Sandbox**: Namespace isolation for shell execution

Input Validation

- Path traversal prevention
- XSS prevention
- SQL injection prevention
- Command injection prevention

Secret Management

- **NEVER** commit secrets to git

- Use `.env` files (mode 0600)
- Rotate on leak (release blocker)

Debugging and Troubleshooting

Common Issues

Build fails:

```
make logo-assets
make build
```

Database connection errors:

```
# Check PostgreSQL running
psql -h localhost -U helix -d helixcode_prod

# Or disable DB
./bin/helixcode --config config/minimal-config.yaml
```

Worker SSH failures:

```
# Verify SSH key authentication
ssh worker@hostname "echo ok"

# Check worker config
cat ~/.helixcode/workers.yaml
```

LLM timeouts:

```
# Check provider status
curl http://localhost:11434/api/tags  # Ollama

# Increase timeout in config
llm.timeout: 60
```

Logging

```
logging:
  level: "debug"  # info, debug, warn, error
  format: "json"  # json, text
  output: "stdout"
```

Health Checks

```
# Server health
curl http://localhost:8080/health

# Database health
```

```
curl http://localhost:8080/health/db

# Redis health
curl http://localhost:8080/health/redis
```

Contributing Guidelines

Before Committing

Run ALL of these:

```
go build ./...
go vet ./...
go test -short ./...
golangci-lint run ./...
grep -rn "simulated\|TODO\|FIXME\|placeholder" internal/ cmd/ &&
        echo "BLUFF FOUND" || echo "clean"
```

Commit Format

```
<type>(<scope>): <summary>

Phase: <phase>
Task: <task-id>
Evidence: <link>

Co-Authored-By: Agent Name <agent@example.com>
```

Types: `feat`, `fix`, `docs`, `test`, `refactor`, `chore`

Pull Request Process

1. Create feature branch from `main`
2. Implement + test + document
3. Run full test suite
4. Update `CONTINUATION.md`
5. Submit PR with evidence of runtime verification

Code Review Checklist

- [ ] All tests pass
- [ ] No hardcoded secrets
- [ ] No simulated/stubbed behavior
- [ ] Documentation updated

- [ ] CONTINUATION.md synced
- [ ] Anti-bluff sweep clean

Resources

- **OpenAPI Spec**: `api/openapi.yaml`
- **Constitution**: `CONSTITUTION.md`
- **AGENTS.md**: `AGENTS.md`
- **Test Strategy**: `ANTI_BLUFF_TESTING_STRATEGY.md`
- **Gap Analysis**: `HELIXCODE_GAP_ANALYSIS.md`

Built with zero-bluff commitment. Every feature actually works.

Sources verified

Sources verified 2026-05-29: https://go.dev/doc/devel/release (Go 1.26.0 GA, latest patch go1.26.3 2026-05-07 — confirms the documented Go toolchain matches CLAUDE.md §3.1), https://endoflife.date/postgresql (PostgreSQL 15 supported through 2027-11 — confirms the "PostgreSQL: 15+" prerequisite is a still-supported floor), https://endoflife.date/redis (Redis 7.4 still receives maintenance/security patches — confirms the "Redis: 7+" prerequisite). Confirmed: golangci-lint as the `go.lintTool`, and the Gin server layer / pgx PostgreSQL layer / go-redis caching match CLAUDE.md §3.1 (Gin v1.11.0, pgx/v5, go-redis/v9). Negative finding: provider API-key examples (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`) are illustrative env-var placeholders, not live endpoints, and per CONST-036 actual model/provider metadata is sourced at runtime from LLMsVerifier (not hardcoded here) — nothing to cross-reference against a provider API doc.